ZIM School - Use Cases

03. Classes: sections, subjects, class periods, teaching loads and the timetable

Modules: Classes
10 use cases (changes) · 6 views and reports
Written from the use case commands, handlers, policies and rules in `src/` on 2026-11-10.

1. What this area does

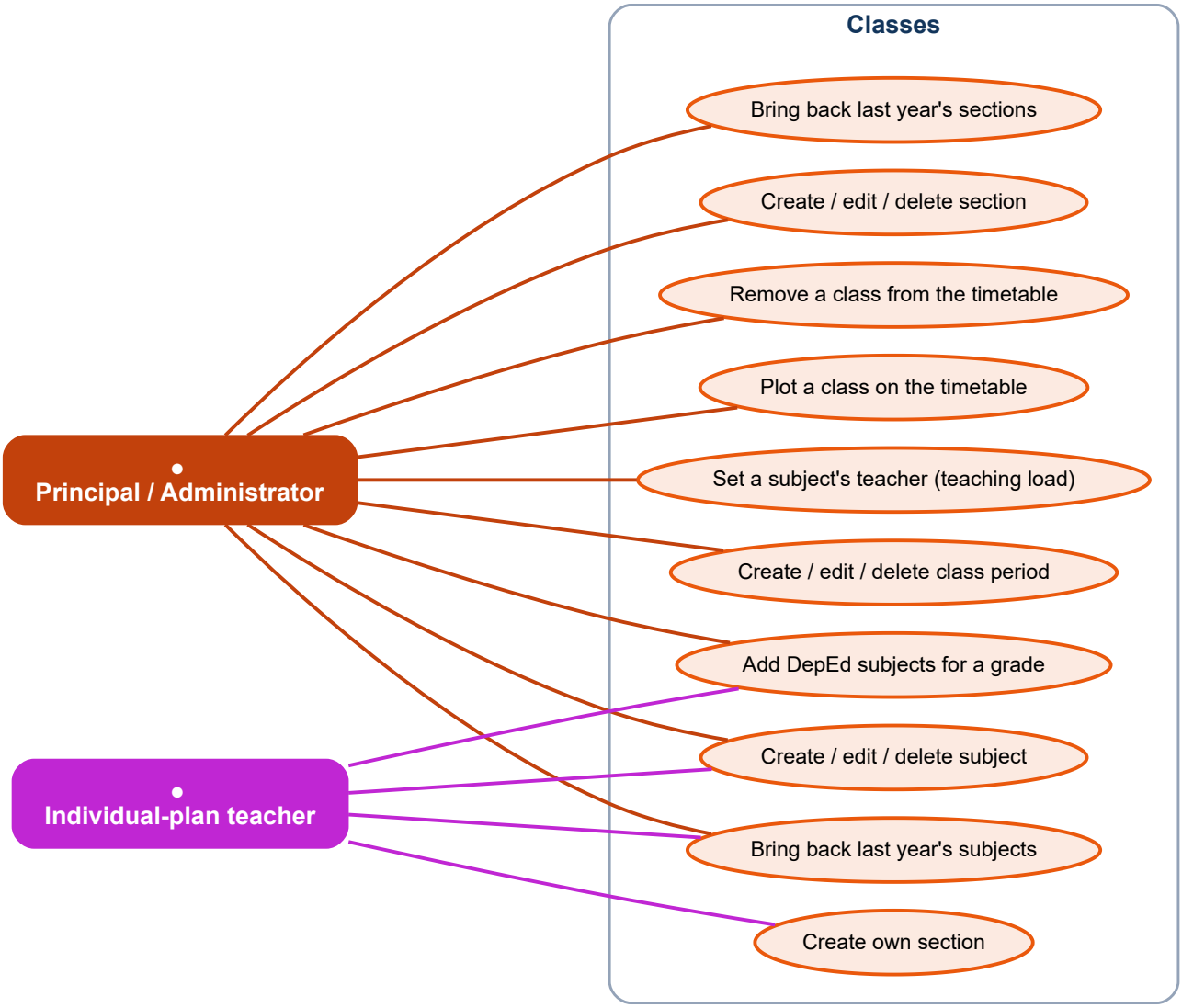
The Classes module answers **what is taught, to whom, by whom and when** in the current school year. The principal (or an administrator) creates the school's **sections** (with an adviser), the **subjects** per grade level (optionally the standard DepEd list, and components such as the four parts of MAPEH) and the **class periods** (the bell schedule). On each section's **timetable** page they pick the teacher of every subject (the teaching load) and place its weekly meetings on the grid.

On the **Individual plan** (no principal) the school's single teacher creates their own section and keeps its subjects from `/teacher/section` and `/teacher/subjects`.

All writes are refused while the school is inactive or its plan expired. Class period and timetable changes take the `school-timetable-<id>` lock so a period can't become a break, or be removed, while a class is being placed in it. The managers check every edit before applying it.

2. Use case map

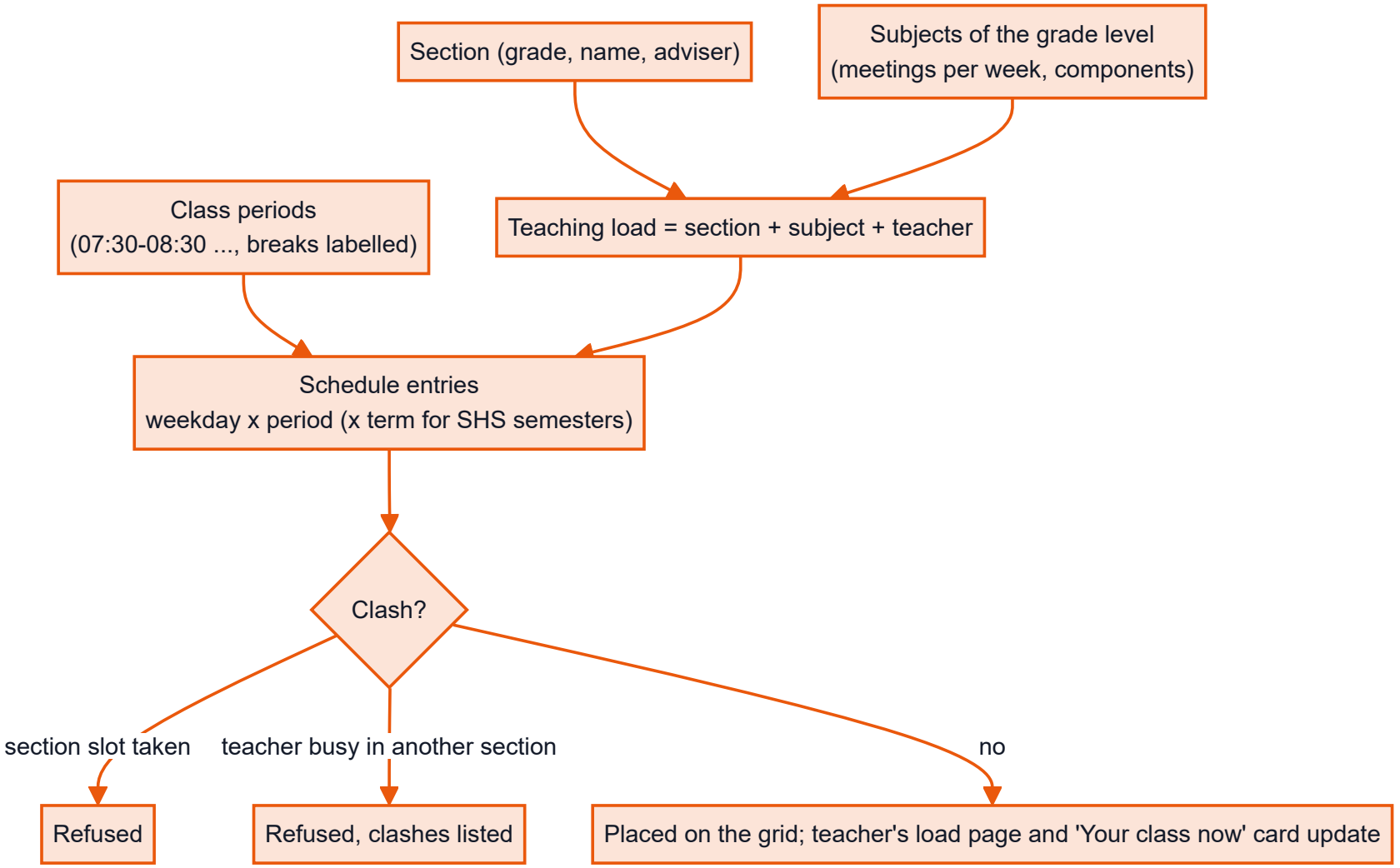
A standard use case diagram: the actors on the left, each use case as an ellipse inside the system frame, and a line for each use case an actor starts. A dashed line has a note on when it applies.



Principal / Administrator Individual-plan teacher Line colour = the actor who starts it.

Classes Ellipse colour = the module the use case belongs to.

3.1 Flow: From section to timetable



Classes Drawn in the colour of the module this flow belongs to.

4. Use cases (changes)

Each card is one use case command sent through the use case bus. It runs in one database transaction: if any rule refuses, nothing is saved and the message in *Alternatives and refusals* is shown to the user.

CLS-01 Create, edit or delete a section

Primary actor	Principal, Administrator
Goal	Keep the school's classes for the current school year.
Where (route)	`school_sections_new / _edit / _delete (/schools/{id}/sections/...)`
Preconditions	• Manages the school; current school year; school active and plan valid.
Main flow	1. The principal chooses grade level, enters the section name and (for Grades 11-12) the track and strand. 2. Optionally picks the adviser from the school's teachers. 3. The system checks the edit and saves it. Delete removes a section that has nothing hanging from it.
Alternatives and refusals	• "This section name already exists in <grade> for this school year." • "Choose the track of this Senior High section." / "Only Grade 11 and 12 sections have a track and strand." • Adviser already advises another section this year (unique adviser per year). • Delete refused: "<section> has N learners. Move or unenroll them first.", "<section> has N pending transfers." or "<section> has a timetable. Remove its classes and teachers first." • Plan blocked: view-only message.
Result (data changed)	Section row created, updated or deleted.
In the code	`CreateSectionCommand, UpdateSectionCommand, DeleteSectionCommand → SectionManager`

CLS-02 Bring back last year's sections

Primary actor	Principal, Administrator
Goal	Copy last year's sections that this year doesn't have yet, without their advisers.
Where (route)	`school_sections_bring_back /schools/{id}/sections/bring-back`
Preconditions	• Manages the school; previous school year exists.
Main flow	1. The Sections page shows the candidates. 2. The principal presses Bring back; missing sections are created for the current year.
Alternatives and refusals	• "There is no previous school year."
Result (data changed)	Section rows (adviser empty).
In the code	`BringBackSectionsCommand`

CLS-03 Create, edit or delete a subject

Primary actor	Principal, Administrator, Individual-plan teacher
Goal	Define the subjects offered to each grade level this school year.
Where (route)	`school_subjects_new / _edit / _delete; teacher_subjects_*`
Preconditions	• The page's scope resolves: the school (principal/admin), or the teacher's own advisory (Individual plan).
Main flow	1. Enter grade level, code, name, position and meetings per week (SHS: semester, track, strand). 2. Optionally make it a component of a parent subject, or give it components (e.g. MAPEH with Music, Arts, PE, Health). 3. The system validates and saves.
Alternatives and refusals	• "The code <code> is already used this school year." / "Use only letters, numbers and dashes in the code." • "Set how many times a week this subject meets." • "Choose the semester of this Senior High subject." • Component rules: "A subject cannot be a component of itself.", "A component must be in the same grade level as its subject.", "A component cannot have components of its own." • "Move or delete the components of <subject> before changing its grade level." • Teacher with a principal: "Your principal sets up the subjects of your school."; without a section yet: "Create your section first."
Result (data changed)	Subject rows (parent_id for components).
In the code	`CreateSubjectCommand, UpdateSubjectCommand, DeleteSubjectCommand → SubjectManager`

CLS-04 Add the DepEd subjects for a grade level

Primary actor	Principal, Administrator, Individual-plan teacher
Goal	Fill a grade level with the standard DepEd subject list in one step.
Where (route)	`school_subjects_deped /schools/{id}/subjects/deped; teacher_subjects_deped`
Preconditions	• The scope resolves; the grade has a built-in list.
Main flow	1. The principal picks a grade and presses Add DepEd subjects. 2. Only subjects the year doesn't have yet are added (with components).
Alternatives and refusals	• "There is no DepEd subject list for this grade level here."
Result (data changed)	Subject rows.
In the code	`AddDepEdSubjectsCommand`

CLS-05 Bring back last year's subjects

Primary actor	Principal, Administrator, Individual-plan teacher
Goal	Copy last year's subjects (with components) that the current year doesn't have.
Where (route)	`school_subjects_bring_back; teacher_subjects_bring_back`
Preconditions	• A previous school year exists.
Main flow	1. The principal presses Bring back on the Subjects page.
Alternatives and refusals	• "There is no previous school year."
Result (data changed)	Subject rows.
In the code	`BringBackSubjectsCommand`

CLS-06 Create, edit or delete a class period

Primary actor	Principal, Administrator
Goal	Keep the bell schedule of the school day.
Where (route)	`school_class_periods_new / _edit / _delete`
Preconditions	• Manages the school; takes the timetable lock.
Main flow	1. The principal enters start and end time; ticks 'break' and gives a label (Recess, Lunch) for non-class slots. 2. The system checks overlaps and saves. Periods are numbered in time order, skipping breaks.
Alternatives and refusals	• "Enter the start time." / "The end time must be after the start time." • "This time overlaps <label> (<range>)." • Turning a used period into a break, or deleting it: "<period> has classes in N sections. Remove them first." • Busy: "Another change is being saved. Please try again."
Result (data changed)	ClassPeriod rows.
In the code	`CreateClassPeriodCommand, UpdateClassPeriodCommand, DeleteClassPeriodCommand → ClassPeriodManager`

CLS-07 Set the teacher of a subject in a section (teaching load)

Primary actor	Principal, Administrator
Goal	Decide who teaches each subject of a section.
Where (route)	`school_timetable_load` `/schools/{id}/sections/{section}/timetable/loads/{subject}`
Preconditions	• Manages the school; the teacher is at the school this year.
Main flow	1. On the section's timetable page the principal picks a teacher for a subject (or clears it). 2. The system creates, changes or removes the TeachingLoad.
Alternatives and refusals	• "This teacher is not in the school this school year." • Clearing while classes are placed: "Remove <subject> from the timetable before clearing its teacher." • Changing teacher where the new one is busy in those slots: refused with each clash listed.
Result (data changed)	TeachingLoad row (and its schedule entries follow the new teacher).
In the code	`SetTeachingLoadCommand → TimetableManager::setTeacher`

CLS-08 Plot a class on the timetable

Primary actor	Principal, Administrator
Goal	Place one weekly meeting of a subject on a weekday and class period (the page uses drag and drop, via the timetable Stimulus controller and JSON calls).
Where (route)	`school_timetable_plot /schools/{id}/sections/{section}/timetable/entries`
Preconditions	• The subject has a teacher and meetings per week; the period is a class period of the school this year.
Main flow	1. The principal drops the subject on a slot. 2. The system takes the timetable lock and checks the section's slot and the teacher's slot. 3. The entry is saved and the grid refreshes.
Alternatives and refusals	• "Pick a teacher for <subject> first." • "Set meetings per week for <subject> on the Subjects page first." • "<break> is a break, not a class period." • "<subject> already has all N meetings this week." • Slot taken or teacher busy elsewhere: "<teacher/section> already has <class> on <day, period>." • A subject with components: "<subject> has components. Pick teachers for its components instead."
Result (data changed)	ScheduleEntry row.
In the code	`PlotClassCommand → TimetableManager::plot`

CLS-09 Remove a class from the timetable

Primary actor	Principal, Administrator
Goal	Take one meeting off the grid.
Where (route)	`school_timetable_remove` `/schools/{id}/sections/{section}/timetable/entries/{entry}`
Preconditions	• The entry belongs to the section.
Main flow	1. The principal removes the meeting; the system deletes it under the timetable lock.
Alternatives and refusals	• "This class is not in the section."
Result (data changed)	ScheduleEntry removed.
In the code	`RemoveClassCommand`

CLS-10 Create own section (Individual plan)

Primary actor	Individual-plan teacher
Goal	Let the only teacher of a school without a principal set up the section they advise.
Where (route)	`teacher_section_new /teacher/section`
Preconditions	• Teacher on a plan without a principal; advises no section yet.
Main flow	1. The teacher fills the section form (no adviser field: they are the adviser). 2. The system creates the section for the current year.
Alternatives and refusals	• "Your principal creates the sections of your school." • "You already advise <section>."
Result (data changed)	Section row with adviser = the teacher.
In the code	`CreateOwnSectionCommand (OwnSectionGate)`

5. Views, downloads and reports (read only)

Each row is one query sent through the query bus. Queries change nothing.

Page / file	Who	Route	What it shows	Query
Sections	Principal, Administrator	`school_sections` `/schools/{id}/sections` `(principal_sections shortcut)`	Sections by grade with enrolled counts and bring-back candidates.	`ListSectionsCommand`, `GetSectionFormCommand`
Subjects	Principal, Administrator, Individual-plan teacher	`school_subjects`; `teacher_subjects`	Subjects by grade (and semester) with components.	`ListSubjectsCommand`, `GetSubjectFormCommand`, `GetSubjectScopeCommand`
Class periods	Principal, Administrator	`school_class_periods`	The school day in time order, with numbering.	`ListClassPeriodsCommand`, `GetClassPeriodFormCommand`
Section timetable	Principal, Administrator	`school_timetable`, `school_timetable_state` `(JSON)`	The weekly grid per term with each subject's teacher and remaining meetings.	`GetSectionTimetableCommand`
Teacher load	Teacher (own), Principal	`teacher_load` `/teacher/load`; `school_teacher_load`	A teacher's loads and weekly schedule (per semester term).	`GetTeacherLoadCommand`
Own section form	Individual-plan teacher	`teacher_section_new`	Whether the teacher may create their section now.	`GetOwnSectionFormCommand`